A Term Paper on

Sequence-Aware Transformer-Based Fraud Detection with LLM-Driven Explainability

— A Graph Neural Network Comparison Study —

Submitted in partial fulfilment of the requirements
for the course *[Course Code]* — Term Paper

of the degree of

Bachelor of Technology in Computer Science & Engineering (Data Science)

Submitted by

Mohd. Shayan Ansari
Enrolment No. A023167024134

Kartik Thakran
Enrolment No. A023167024129

Under the guidance of

Aakanshi Agrawal
[Designation]

Amity School of Engineering and Technology (ASET)
Amity University, Noida

July 2026

DECLARATION

We, Mohd. Shayan Ansari and Kartik Thakran, declare that this term paper titled "Sequence-Aware Transformer-Based Fraud Detection with LLM-Driven Explainability" is a genuine record of our own work, carried out under the guidance of Aakanshi Agrawal at the Amity School of Engineering and Technology (ASET), Amity University, Noida, towards the course *[Course Code]* of the B.Tech, Computer Science & Engineering (Data Science) programme.

The idea, the design of the models, the experiments, and the results presented in the following pages were developed and run by us. Wherever we have drawn on the ideas, methods, datasets, or published work of others, we have named the source and cited it at the appropriate place. The two of us contributed jointly to the reading, the coding, the analysis, and the writing of this report.

We also state that this work has not been submitted, in full or in part, to this institution or to any other, for the award of any degree, diploma, or similar qualification. To the best of our knowledge, it contains no material previously published or written by another person except where due reference is made.

Finally, we note that the entire project was built with free and open-source tools and a publicly available dataset; no paid service or proprietary resource was used at any stage.

Place: Noida

Date: *[date to be filled at submission]*

Mohd. Shayan Ansari	**Kartik Thakran**
Enrolment No. A023167024134	Enrolment No. A023167024129

CERTIFICATE

This is to certify that the term paper titled "Sequence-Aware Transformer-Based Fraud Detection with LLM-Driven Explainability" is a bona fide record of the work carried out by Mohd. Shayan Ansari (Enrolment No. A023167024134) and Kartik Thakran (Enrolment No. A023167024129) under my supervision, in partial fulfilment of the requirements for the course *[Course Code]* — Term Paper of the B.Tech, Computer Science & Engineering (Data Science) programme at the Amity School of Engineering and Technology (ASET), Amity University, Noida.

The students carried out this work themselves during their period of study. The analysis, the models, and the results presented in the report are the outcome of their own effort, and any external material they relied upon has been acknowledged and cited. To the best of my knowledge, neither this report nor any part of it has been submitted to this or any other institution for the award of any degree or diploma.

I am satisfied with the quality and sincerity of their work, and I wish them well in their future academic and professional pursuits.

Place: Noida

Date: *[date to be filled at submission]*

_____

Aakanshi Agrawal

[Designation]

Amity School of Engineering and Technology (ASET)

Amity University, Noida

ACKNOWLEDGEMENT

We are sincerely grateful to our guide, Aakanshi Agrawal, for her steady advice, patience, and the hard questions that pushed this work forward at every stage. We also thank the faculty of the Amity School of Engineering and Technology (ASET), Amity University, Noida, and our families and friends, for their support through the many evenings this project demanded. Finally, we are grateful for the free and open-source tools and the public IEEE-CIS dataset that made this project possible without cost.

Mohd. Shayan Ansari

Kartik Thakran

ABSTRACT

Financial fraud hides inside a tiny fraction of everyday transactions, which makes it both costly and hard to catch. In this term paper we study fraud detection on the IEEE-CIS dataset, where only about 3.5% of transactions are fraudulent, and we ask two questions at once: can a model flag fraud accurately, and can it explain why in plain language? Our approach looks at each payment from two angles. A Transformer reads the recent sequence of a card's transactions to capture how its spending behaves over time, while a graph neural network looks at the web of shared cards, devices, merchants, and regions to capture the relationships between transactions. We fuse these two views into a single fraud score. To see which view matters, we compare four models — a Transformer on its own, and the Transformer combined in turn with GraphSAGE, GAT, and a heterogeneous spatio-temporal network — on the same time-aware split. Every graph-augmented model beats the sequence-only baseline, and the Transformer-with-GAT model does best, though its lead over GraphSAGE is slim; adding relationship structure helped more than the exact way it was aggregated. On top of the detector we build an explainability layer: SHAP attributes each decision to input features, and a local, offline language model turns those numbers into a grounded, plain-English explanation a bank officer could read. The whole system was built with free, open-source tools at zero cost.

Keywords: fraud detection; class imbalance; Transformer; graph neural networks; GraphSAGE; GAT; embedding fusion; explainable AI; SHAP; local large language models.

TABLE OF CONTENTS

Chapter 1 Introduction

Picture a card that has only ever bought groceries and paid a phone bill suddenly making two large purchases in a city it has never visited — nothing about that one transaction looks wrong on its own. Or picture a single mobile device quietly used to pay with five different people's cards in one week; each payment looks perfectly ordinary by itself. In both cases the giveaway is not the transaction but the pattern around it — the card's own history, or the connections it shares with other transactions — and a model that studies one row of a spreadsheet at a time simply cannot see either kind of clue.

To catch both kinds of clue, we look at every transaction two ways at once. A Transformer — the same kind of model behind modern language models — reads a card's transactions in the order they happened, learning what normal spending looks like so it can notice when the rhythm breaks. A graph neural network instead treats transactions, cards, devices, merchants, and regions as a network of connections, and passes information along shared links so a suspicious neighbour can raise a flag on transactions that would otherwise look clean. We turn each view into a compact 128-number description and fuse the two into a single fraud probability.

Put simply, the Transformer learns the when and how of a card's own behaviour over time — is this spending pattern normal for this card, is the timing unusual — call it the temporal signal. The graph instead learns who is connected to whom — is this transaction linked to a device that many different cards have used — call it the structural signal. Fusing the two means a transaction can be flagged because it looks unusual for that card, or because it looks suspiciously connected to other fraud, or both, which is exactly the two-sided blind spot the examples above pointed to. We then explain the fused score in plain English using SHAP and a small, locally run language model. Figure 1.1 sketches the design.

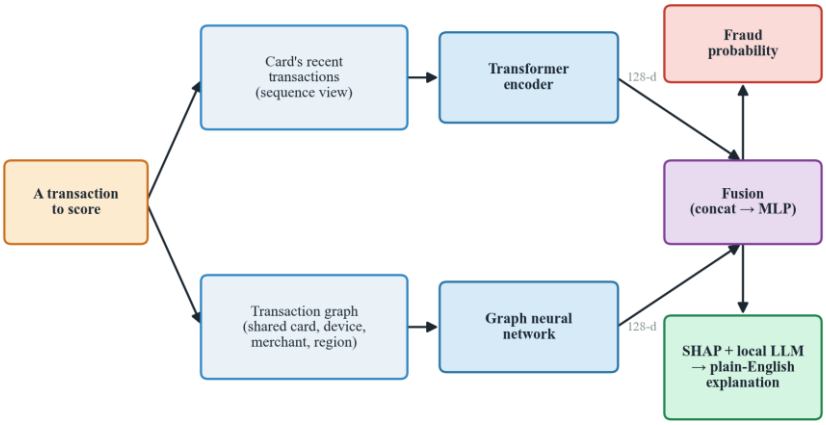

Figure 1.1: The high-level design. The same transaction is read as a sequence (by a Transformer) and as part of a graph (by a graph neural network); the two 128-d summaries are fused into a fraud probability, which is then explained with SHAP and a local language model.

1.1 Problem statement

Our testbed is the IEEE-CIS fraud dataset, a large, real-world collection of card transactions in which only about one in thirty is fraudulent. This imbalance is the heart of the challenge: a model that calls everything legitimate is right most of the time and useless in practice. Our problem is therefore twofold — build a detector that finds fraud well despite this imbalance, using both a card's behaviour over time and the relationships between transactions, and make each decision understandable enough that a reviewer can see why a payment was flagged rather than being handed a bare number.

1.2 Objectives

With that problem in mind, we set four goals: build the two views (sequence and graph) from the raw data without letting future information leak into the past; design a fusion model that learns from both views together; honestly test whether the graph view helps by comparing four model variants — a Transformer alone, and the Transformer paired with GraphSAGE, GAT, and a heterogeneous spatio-temporal network — under identical conditions; and attach an explanation layer that reports the real reasons behind each prediction, not invented ones. Every part of this had to be built with free, open-source tools, at zero cost.

Chapter 2 Background and Literature Review

Two model families do the heavy lifting in this report, so it is worth saying plainly what each one is. A Transformer is a neural network built to make sense of ordered data — originally sentences, but here a run of a card's own transactions in the order they happened. Its key idea, called attention, lets each transaction look back over the earlier ones and lean more heavily on whichever are most relevant, instead of blurring everything into one running memory the way older sequence models did. We train our Transformer from scratch on transaction data — it is not a pretrained language model such as BERT, and it never reads any text — so that it learns a card's normal spending rhythm and notices when a new transaction breaks it. Figure 2.1 gives the idea.

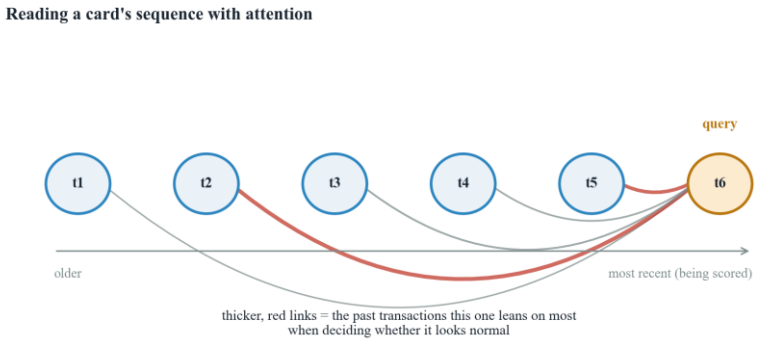

Figure 2.1: Attention over a card's transaction sequence. The transaction being scored (the query) forms its view by weighing the card's earlier transactions, leaning on the ones that best explain whether the current spend is normal.

A graph neural network instead works on relationships. A graph is simply a set of nodes joined by links: here, a transaction node connects to the card that made it, the device it came from, the merchant, and the region. Its core idea, message passing, lets each node gather information from the neighbours it is linked to and update its own description; after a couple of rounds, that information has travelled several hops. This is what lets a transaction that looks clean on its own inherit suspicion from, say, a device it shares with several already-flagged payments — the fraud-ring pattern from Chapter 1. Figure 2.2 shows one transaction pulling in messages from its neighbours. We compare three such networks that differ mainly in how a node weighs its neighbours' messages: GraphSAGE, which samples and aggregates; GAT, which learns an attention weight per edge, borrowing the same idea that makes the Transformer work; and a

heterogeneous, time-aware network built for the different kinds of node and edge in our graph.

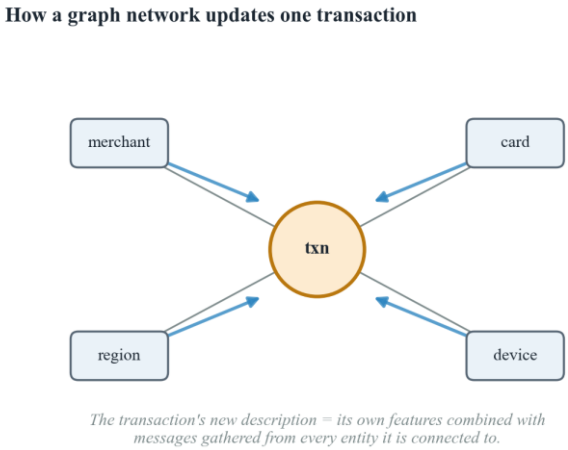

Figure 2.2: Message passing. A transaction node updates its description by gathering messages from the card, device, merchant, and region it is connected to, so evidence can travel between related transactions.

Pairing a Transformer with a graph network for fraud detection is not our idea alone. The closest relative, and the base paper we started from, is STA-GT (Tian & Liu, 2023), which builds a heterogeneous transaction graph with a temporal-encoding step and a transformer module, and reports gains over several GNN baselines. Our design differs in three ways: we run a Transformer directly over each card's own sequence rather than folding time into the graph, we run a controlled four-model comparison to measure what the graph view is actually worth, and we add a plain-English explanation layer, which STA-GT itself leaves as future work. Similar transformer-plus-graph ideas appear elsewhere too — RAGFormer (2024) fuses the two views through attention on the same reasoning that semantic and structural information are complementary — which tells us this is a sound direction, not an isolated choice.

Taken together, these works show that transformer-and-graph fraud models are strong but rarely stop to measure, under matched conditions, how much the graph actually adds, and explainability, when present, is usually an afterthought. Our project addresses that gap at a modest, undergraduate scale: a genuine sequence view fused with a relationship graph, tested through a clean four-model comparison, with every prediction finished off by a grounded, plain-English explanation.

Chapter 3 Methodology

This chapter explains, in plain terms, how raw transaction rows become the two views, the four models built on them, and how we trained and judged each one.

3.1 Dataset and the two views

We work with the IEEE-CIS fraud dataset from Kaggle: 590,540 transactions joined with device and identity details, of which only about 3.5% are fraud. A block of columns (the C, D, and V families) are anonymised — Vesta engineered them from real behaviour but does not say what they mean, so we use them without a human-readable label, a limitation that returns in Chapter 4. The dataset has no user field, so we build a stand-in "client id" from the card and address fields that tend to stay constant for one holder, which is what lets us read a card's behaviour over time at all. Cleaning was simple: we dropped columns missing in over 90% of rows, filled the rest (median for numbers, "unknown" for categories), and added a few engineered features — hour and day from the transaction time, a log-scaled amount, and a running per-card spending average computed only from past transactions, so nothing about a transaction's own outcome leaks backwards.

We split the data by time rather than at random — the earliest 70% for training, the next 15% for validation, and the last 15% for testing (413,378 / 88,581 / 88,581 transactions) — so a model is always tested on transactions that happened after everything it learned from, the way fraud detection really works. The fraud rate drifts slightly across the splits (about 3.5%, 3.4%, and 3.5%), an honest side effect of splitting by time. The feature scaler and category encodings were fit on the training split only.

Before any model sees the data, we reshape the same cleaned table into two complementary structures. For the sequence view, we group transactions by client id, order them by time, and pair each one with a window of up to its previous nineteen transactions (padded and masked where a card's history is shorter), so a Transformer can read how a card has been behaving. For the graph view, we build a heterogeneous graph: every transaction, card, device, merchant, and region becomes a node (590,540 transaction nodes, 42,946 cards, 1,943 devices, 5 merchants, 332 regions), linked by typed edges such as made-by or on-device, so that transactions sharing a card, device, merchant, or region become neighbours. Figure 3.1 shows this graph. The two views

are kept in the same row order throughout, so a transaction's sequence window and its graph node always describe the same payment.

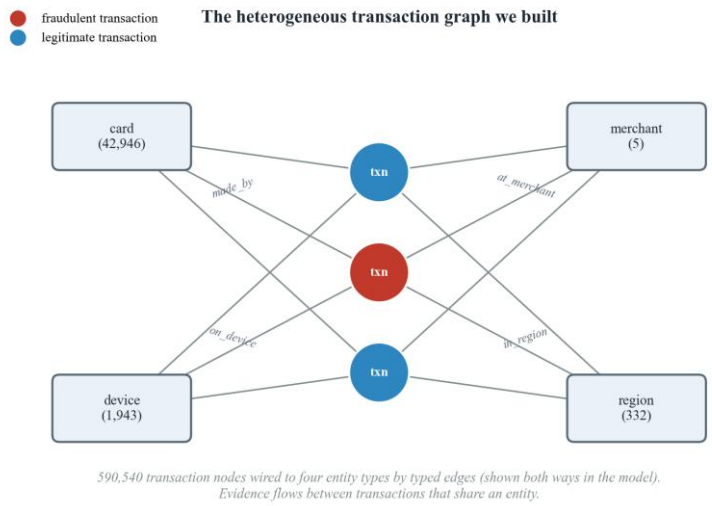

Figure 3.1: The heterogeneous graph we constructed from the CSV. Transaction nodes are connected by typed edges to four kinds of entity node (with their counts). Two transactions that share, say, a device become neighbours, so evidence can pass between them.

3.2 The Transformer branch

Before Transformers, sequence models such as RNNs and LSTMs were the default choice for anything ordered — text, speech, time series — but they process one step at a time and tend to forget what happened many steps back, since everything has to pass through a single running memory. The Transformer replaces that with self-attention, letting every position look directly at every other position regardless of distance, which handles long-range dependencies better and is far easier to parallelise during training. It is the architecture behind most of today's large language models (GPT, BERT, and similar), and it has since spread well beyond text into vision, speech, and tabular or time-series problems — which is the setting we borrow it for here: not sentences, but a card's ordered transaction history.

The sequence branch is a small Transformer, the same attention-based architecture described in Chapter 2, but trained from scratch on transaction windows rather than text. It turns each 32-feature transaction into a 128-number vector, adds a position signal so order is not lost, and passes the window through two attention layers that ignore the padded positions. We take the model's description of the most recent (real) transaction in the window as the branch's 128-number output — a summary of

how the card has been behaving. We kept this model small on purpose: the windows are short and the data is large, so a heavier model would only cost training time.

3.3 The three graph branches

Graph neural networks are used wherever data naturally forms a network rather than a flat table — recommending products on a shared purchase graph, screening molecules for drug discovery, ranking web pages, and, as here, spotting fraud rings through the accounts or devices they share. The three branches we compare sit at different points on that family's spread: GraphSAGE is the efficient, inductive workhorse that scales to large graphs by sampling neighbours; GAT adds learned attention over edges, trading some speed for the ability to weigh neighbours unevenly; and heterogeneous networks go further still, keeping separate rules for different node and edge types instead of treating the whole graph as one uniform structure. Comparing all three lets us see whether that extra sophistication is worth its cost on our data.

GraphSAGE is the simplest graph branch: each node gathers its neighbours' messages and combines them through one learned rule that treats every neighbour of a given type the same way, so a transaction comes to reflect the cards, devices, merchants, and regions around it without singling any one of them out. We run this over two layers, one rule per edge type, and add the results together.

GAT changes exactly that one thing: instead of treating every neighbour the same, it learns an attention weight for each edge, so a transaction can lean much more on its most telling connection — a device shared with several flagged cards, say — and largely ignore an ordinary one, the same idea that makes the Transformer work. One honest caveat: we had to cut GAT's attention heads from four to one because the free Colab GPU we trained on ran out of memory on the full graph, so we treat its results as a lower bound.

The third branch is a heterogeneous spatio-temporal network, and it is worth unpacking what each half of that name means. "Spatio" is about structure: rather than treating every edge the same way, or even just weighting them as GAT does, it keeps a genuinely separate message-passing rule for each kind of connection — one for card edges, one for device edges, one for merchant edges, and so on — so the model can learn that a shared device means something different from a shared region. "Temporal" is about time: alongside that structure, we give each transaction node its hour, day, and

position in the overall chronological order, so the model can notice not just who a transaction is connected to, but when. Together, this is meant to catch a pattern neither GraphSAGE nor GAT can see on its own — say, five cards suddenly sharing one device within the same two-day window, where the sharing alone might look ordinary but the sudden timing gives it away. In principle this branch can express more than the other two — in practice, as Chapter 5 shows, that extra machinery did not pay off on our data.

3.4 Fusion, training, and evaluation

Each transaction ends up with two 128-number summaries — one from the Transformer, one from a graph branch — and fusion simply joins them: concatenate into 256 numbers, mix them back down to 128 with a small layer, and squeeze to a single fraud probability (Figure 3.2). The sequence-only baseline, Model 1, is the same head with the graph half removed, which is what lets us measure exactly what the graph adds.

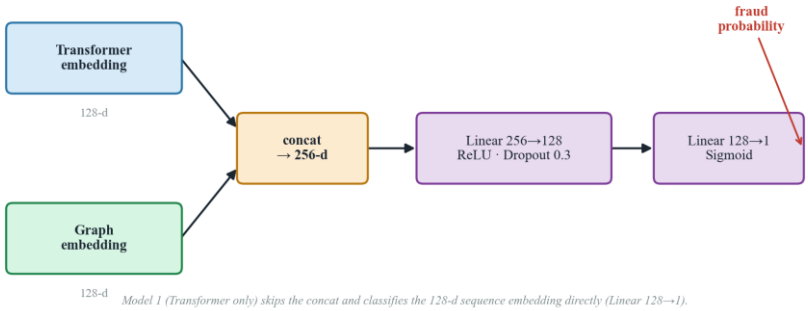

Figure 3.2: The fusion classifier. The Transformer and graph embeddings are concatenated and passed through a small multi-layer perceptron that outputs a fraud probability. Model 1 skips the concatenation and classifies the sequence embedding alone.

All four models were trained the same way, on a free Google Colab GPU, so the comparison stays fair: Adam optimiser, batches of 256, up to fifty passes with early stopping on validation performance. Because fraud is rare, we used a focal loss that pays more attention to the hard, fraudulent minority instead of an ordinary loss a model could win by ignoring fraud altogether. For evaluation, plain accuracy is misleading here — a model that flags nothing still scores about 97% — so we lead with PR-AUC, a single number that captures how well a model ranks fraud cases ahead of legitimate ones, alongside precision, recall, F1, and ROC-AUC. Chapter 5 reports all of this across the four models.

Chapter 4 Explainability and System Design

A model like ours is often called a black box for good reason: it can tell us that a transaction is 75% likely to be fraud, but not why. We know what it decided; the harder and more useful question is why it decided that. A bare score is hard to act on — a reviewer handed just "75% likely fraud" has no way to check it, question it, or explain it to a customer. This chapter describes the two-step explanation layer we built to answer that question, and the small application that wraps the detector into something clickable.

4.1 Attributing a decision with SHAP

SHAP takes a single prediction and divides it among the input features, handing each a signed number for how much it pushed the score up or down. We apply it to the same 32 features the sequence branch reads, using the model-agnostic Kernel variant with a background of 100 training transactions to represent "an ordinary payment." Because re-running the graph network for every SHAP sample would be too slow, we hold the transaction's graph embedding fixed and let SHAP vary only its own features — fast enough to run on a plain laptop. Figure 4.1 shows a worked example: a transaction the model correctly flagged as fraud with 75% confidence, where a handful of anonymised count features (C1, C11, C8, C12) do almost all of the pushing, and the contributions sum back exactly to the model's score. The one honest catch is that these top features are anonymised — we can say C1 mattered most and by how much, but not what it means.

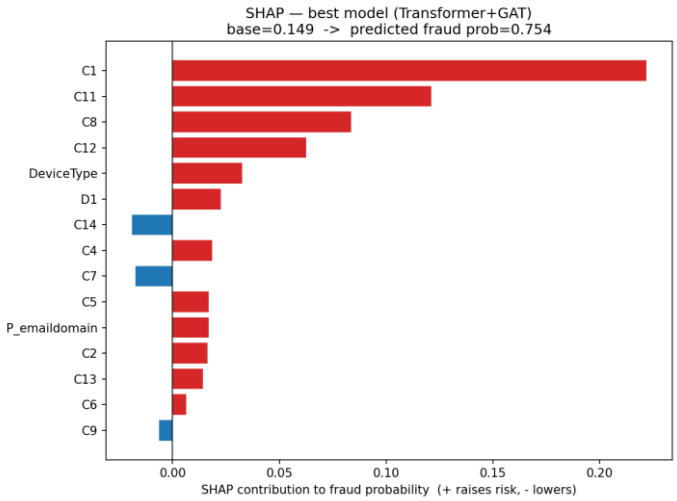

Figure 4.1: SHAP attribution for one fraud transaction (predicted probability 0.75). Red bars raise the fraud score and blue bars lower it; the baseline plus all contributions sum back to the model's output. A few anonymised count features (C1, C11, C8, C12) dominate the decision.

4.2 Turning the numbers into words with a local language model

The second step hands SHAP's ranked list to a small language model and asks it to write a few plain sentences a bank officer could read, using only the given evidence and inventing nothing. It runs locally through Ollama — no API key, no cost, nothing leaves the machine. We settled on llama3.2:3b after phi3 tended to embroider (inventing timing or location details, or misreading scaled values as percentages); with a tightly worded prompt, llama3.2:3b stays grounded, correctly naming the real factors and their real weights. We checked the pipeline on three fraud and two legitimate transactions and confirmed, for every case, that the SHAP contributions sum exactly to the model's score and that the explanation's direction (risk up or down) matches the outcome. One caveat: a free, three-billion-parameter model can still be a little chatty, so its paragraphs are best read as a helpful gloss on the SHAP numbers rather than an authority in their own right.

4.3 The demonstration application

To make the system tangible we built a small Streamlit web app with three pages: a live demo that scores a transaction and shows its SHAP factors and LLM explanation together (Figure 4.2), an analysis page summarising the four-model study, and a reproducibility page listing the artifacts and steps to rebuild the project. The demo accepts a transaction via a sample click, a custom form, or a small batch upload. Two honest limitations: a custom, typed-in transaction cannot carry the rich per-card history the model trained on, so its prediction is more a demonstration than a calibrated risk figure; and since free Streamlit hosting cannot run a local LLM, the plain-English explanation is a local-only feature — a hosted version would fall back to a templated summary of the same SHAP factors.

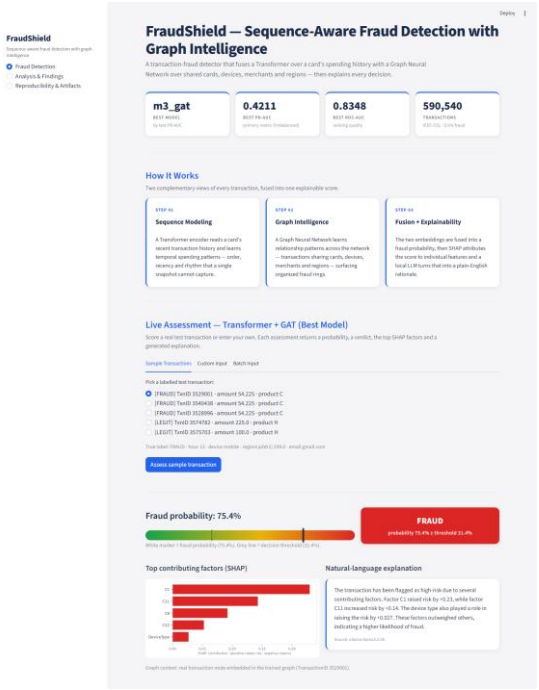

Figure 4.2: The live-assessment console, scoring the same fraud transaction (TransactionID 3529001) walked through in Figure 4.1 — probability, verdict against the decision threshold, the SHAP factor chart, and the local-LLM explanation together on one screen.

Chapter 5 Results and Discussion

This chapter reports what the four models did on the held-out test set and answers our central question: does adding a graph view to a sequence model actually help catch fraud?

5.1 The four-model comparison

Table 5.1 collects the test-set results. We lead with PR-AUC, the metric that matters most when only about one transaction in thirty is fraud (F1, precision, and recall are measured at the usual 0.5 threshold; accuracy is left out since every model scores about 0.971 simply by calling almost everything legitimate). The two graph-augmented models, GraphSAGE and GAT, both beat the sequence-only baseline — the outcome the comparison was built to test — though the differences are small: PR-AUC ranges only from 0.409 to 0.421.

Table 5.1: Test-set performance of the four models (F1, precision, and recall at the 0.5 threshold). The Transformer + GAT model wins on PR-AUC, the primary metric; both graph models beat the sequence-only baseline.

Model	Architecture	PR-AUC	ROC-AUC	F1	Precision	Recall
1	Transformer only (baseline)	0.4142	0.8270	0.3205	0.8743	0.1962
2	Transformer + GraphSAGE	0.4198	0.8341	0.3538	0.7848	0.2283
3	**Transformer + GAT**	**0.4211**	**0.8348**	**0.3726**	**0.8133**	**0.2416**
4	Transformer + ST-HGNN	0.4093	0.8210	0.3341	0.8612	0.2073

Figure 5.1 shows the same story at a glance, and makes the recall problem plain — every model's recall sits low, near 0.2.

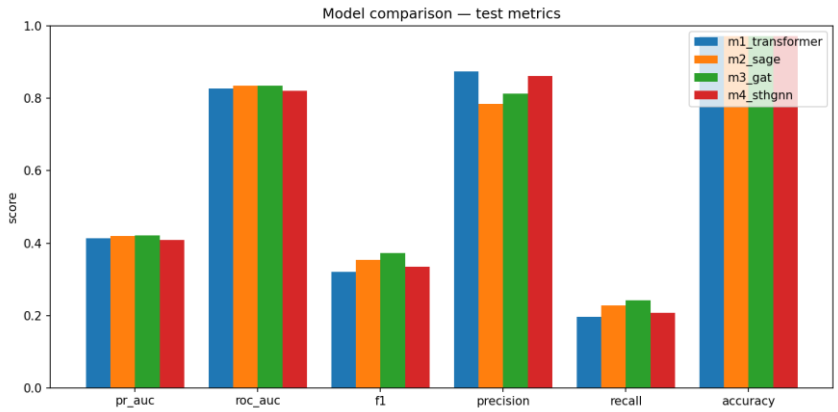

Figure 5.1: The six test metrics side by side for all four models. Accuracy is uniformly high and uninformative; recall is uniformly low; the models separate most on F1 and, slightly, on PR-AUC.

5.2 The ranking curves

Figure 5.2 and Figure 5.3 show the ROC and precision-recall curves. All four models overlap tightly — no curve pulls decisively ahead — which tells us the choice of graph model matters far less than having a graph at all. Every model sits well above the 0.035 base-rate line, so all four are doing real work; precision holds up well until recall passes about 0.4, after which it drops away, the shape of a detector that can be trusted on its most confident flags but cannot reach most of the fraud without more false alarms.

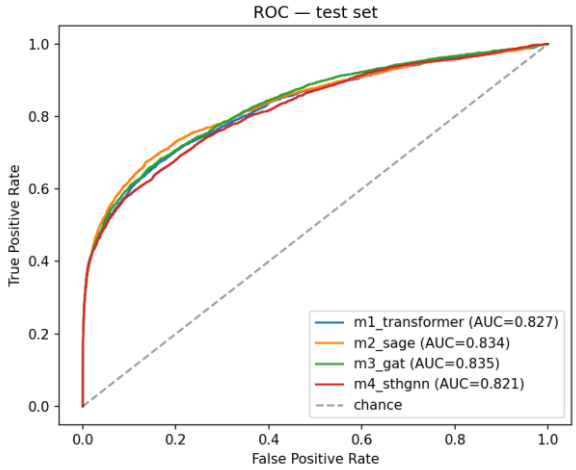

Figure 5.2: ROC curves on the test set. All four models track closely, well above the chance diagonal; GAT and GraphSAGE sit marginally highest.

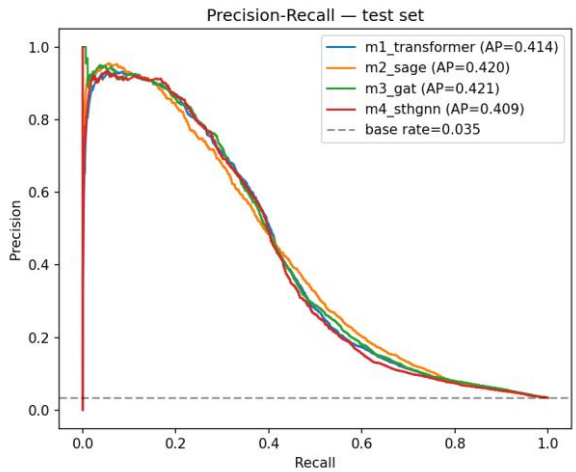

Figure 5.3: Precision-recall curves on the test set. The models overlap heavily and all sit far above the 0.035 base rate, but precision drops steeply once recall passes roughly 0.4.

5.3 Reading the results model by model

Model 1, the sequence-only baseline, has the highest precision of any model (0.874) but the lowest recall (0.196) — and is the fastest to run. It is a reasonable choice if speed mattered more than a fraction of PR-AUC, but here the graph models beat it.

Model 2 (GraphSAGE) is the first sign the graph carries real signal: PR-AUC rises to 0.420 and recall to 0.228, at the cost of some precision (0.785) — a wider net that catches more fraud along with more false alarms.

Model 3 (GAT) is the overall winner, with the best PR-AUC (0.4211), F1 (0.373), and recall (0.242), likely because it learns to weigh each neighbour instead of averaging them evenly. Its lead over GraphSAGE, however, is a razor-thin 0.0013 in PR-AUC — well within noise — so we treat GAT and GraphSAGE as roughly tied rather than crown GAT a clear winner.

Model 4 (ST-HGNN), the heaviest and most elaborate model, is the most interesting result precisely because it disappoints: it finishes last on PR-AUC (0.409), likely because its extra temporal machinery duplicated what the Transformer already captured while being harder to train within our free-GPU budget — a genuine finding, not a failure.

Stepping back, the graph clearly helps — but between GAT and GraphSAGE specifically, which one wins is a closer call than it looks: their gap here is razor-thin. It is worth being explicit about why that might be, rather than just noting that it is close. GraphSAGE, GAT, and heterogeneous networks are built for different strengths — GraphSAGE for efficiently aggregating large, fairly uniform neighbourhoods, GAT for picking out the few neighbours that matter much more than the rest, and heterogeneous networks for graphs where different node and edge types genuinely behave differently. Our transaction graph may simply not stress-test those differences very hard, which is plausibly why all three land so close together. On a different dataset — one that leaned harder into one of those strengths — the gap could easily have been far wider than what we see here, not just a different winner by a similarly small margin. The safer takeaway is therefore not “GAT is the best graph model,” but “adding a graph view helps, and which graph architecture wins, and by how much, depends on the data.” ST-HGNN is a partial exception to this story: its gap behind the other two here is large enough, and explainable enough, to be a real finding rather than noise. A second pattern worth naming is the high-precision, low-recall shape shared by every model, a hallmark of

severe imbalance — which is why the best-F1 thresholds all fall well below 0.5, between about 0.24 and 0.31.

5.4 What the winning model catches and misses

Figure 5.4 shows the winning model's confusion matrix at its best-F1 threshold (0.314). Of 3,083 genuine frauds it catches 1,155 and misses 1,928 — a recall of about 37% — while raising 900 false alarms among 85,498 legitimate payments. It is trustworthy when it stays quiet, but it still lets most fraud through, which is why a system like this belongs in front of a human review queue rather than acting alone.

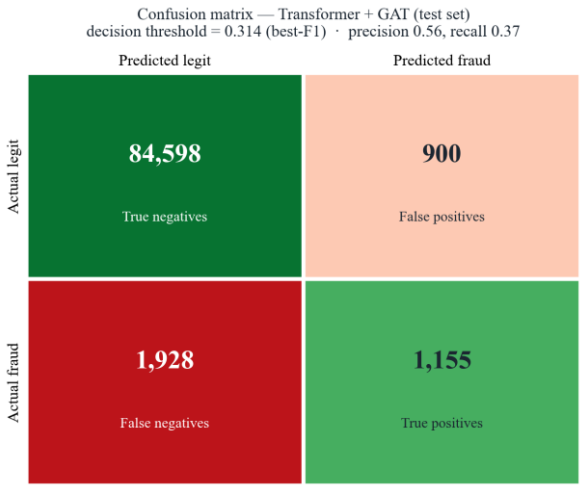

Figure 5.4: Confusion matrix for the winning Transformer + GAT model on the test set, at its best-F1 threshold (0.314). It catches 1,155 of 3,083 frauds while raising 900 false alarms among 85,498 legitimate transactions.

5.5 An example decision, explained

To see the explanation layer at work, take the fraud transaction from Chapter 4, scored 0.75 by the winning model. Passed through SHAP and then the local language model, that score becomes a short, grounded note:

> *"The transaction is flagged as high-risk due to several key factors. Factor C1 raised risk by +0.22, while factor C11 increased risk by +0.12. These two factors, along with factor C8 (+0.08) and factor C12 (+0.06), all contribute to the overall high-risk assessment. The device type also played a role in raising the risk by +0.033."*

The note names the real factors and their real weights and invents nothing — though because the top drivers are anonymised codes, it can say C1 mattered most without saying what C1 represents.

5.6 Limitations

Several limits should temper these numbers. The GAT and ST-HGNN models were trained with a single attention head instead of four, since the free Colab GPU ran out of memory on the full graph — both are running below their potential, so GAT's numbers, and its narrow lead over GraphSAGE, should be read as a lower bound rather than the final word. Beyond that: our explanations can rank and quantify a decision's drivers but not always name them, since many features are anonymised; the graph is transductive, computed over all transactions at once rather than in a live streaming setting; and a PR-AUC near 0.42 and recall near 0.37, while respectable for this level of imbalance, are not figures that would let a model run unsupervised. These limits point directly at the future work of the final chapter.

Chapter 6 Conclusion and Future Work

We set out to do two things: catch fraud in a badly imbalanced stream of transactions, and explain each decision in language a person could read. Our design followed from a simple observation — fraud rarely lives in a single row, but in how a card behaves over time and how transactions connect through shared cards, devices, merchants, and regions — so we read each payment as a sequence with a Transformer, again as part of a graph with a graph neural network, fused the two into one score, and traced that score back to its reasons.

6.1 What we found

Adding a graph view to the sequence model helps: both graph-augmented models beat the sequence-only baseline on PR-AUC, and the GAT fusion narrowly took the top spot. But the gap between GAT and GraphSAGE was razor-thin — and because each architecture is built for a different kind of strength (efficient aggregation versus learned attention versus per-type rules), that gap could easily have been much wider, in either direction, on a dataset that leaned harder into one of those strengths. So the graph mattered far more than which graph architecture we picked, and the ranking itself — including how far apart the models end up — is the kind of thing that depends on the data rather than one model being universally best. The heaviest model, the heterogeneous spatio-temporal network, actually finished last, a reminder that more machinery is not the same as more signal. On the explainability side, SHAP paired with a small local language model produced explanations that were faithful under two objective checks, built entirely with free, open-source tools.

6.2 Limitations

A few honest limits are worth repeating: GAT and the spatio-temporal model were trained with a single attention head instead of four due to free-GPU memory limits, so their numbers are a lower bound; many features are anonymised, so our explanations can rank a decision's drivers but not always name them; and the graph is transductive, built over all transactions at once rather than a live stream.

6.3 Future work

The natural next steps are to retrain GAT and the spatio-temporal model with full attention heads on stronger hardware to see if the ranking holds, let the graph carry its own sense of time rather than leaning on the Transformer for it, calibrate the demo's custom-input path against sensible defaults, and move toward a genuine deployment — a hosted service with a human review queue and, where privacy and cost allow, the local language model still explaining every flag. For an undergraduate project built entirely for free, that is a fair place to stop, and a solid base for the next version.

References

The list below gives the sources cited in the report. Method and base papers are named where each idea is first introduced; the dataset and the free tools we relied on are listed at the end. Entries follow an author-year style and are ordered alphabetically.

Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

Financial fraud detection using graph neural networks: A systematic review. (2023). *Expert Systems with Applications*.

Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)* (Vol. 30).

IEEE-CIS Fraud Detection. (2019). *Kaggle competition dataset, in partnership with Vesta Corporation*. https://www.kaggle.com/c/ieee-fraud-detection

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)* (Vol. 30).

Ollama. (2023). *Ollama: Run large language models locally* [Computer software]. https://ollama.com

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., … & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)* (Vol. 32).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

RAGFormer: Learning semantic attributes and topological structure for fraud detection. (2024). *arXiv preprint* arXiv:2402.17472.

Streamlit Inc. (2019). *Streamlit: A faster way to build and share data apps* [Computer software]. https://streamlit.io

Tian, Y., & Liu, G. (2023). Transaction fraud detection via spatial-temporal-aware graph transformer. *arXiv preprint* arXiv:2307.05121.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)* (Vol. 30).

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. In *International Conference on Learning Representations (ICLR)*.

Appendix

This appendix collects two things that support the chapters above without interrupting their argument: the exact hyperparameters shared by every model in the comparison, and two further views of the demonstration application that the main text did not have room to show.

Appendix A: Hyperparameters

Table A.1 lists every hyperparameter defined once in config.py and reused by all four models, so that the study stays a controlled comparison of architecture alone. The one deliberate exception is the number of attention heads in GAT, reduced from the design's original four to one — a hardware constraint from the free-tier Colab T4 GPU running out of memory over the full transaction graph, not an oversight, and noted again here because it is the single most important caveat on the GAT and ST-HGNN results in Chapter 5.

Table A.1: Global hyperparameters, shared by all four models unless noted.

Hyperparameter	Value	Role
Random seed	42	Fixed across all runs for reproducibility
Embedding dimension	128	Output width of both the Transformer and GNN branches
Sequence length	20	Transactions kept per card (padded/truncated)
Transformer layers	2	Encoder depth of the sequence branch
Transformer attention heads	4	Self-attention heads per encoder layer
Transformer feed-forward dim	256	Width of the encoder's feed-forward block
GNN hidden dimension	128	Hidden width in GraphSAGE / GAT / ST-HGNN
GNN layers	2	Message-passing depth in every graph branch
GAT attention heads	1 (reduced from 4)	Colab T4 memory constraint; see note above
Dropout	0.3	Applied after every branch and in the fusion MLP
Batch size	256	Mini-batch size for training
Learning rate	1e-3	Adam optimiser
Weight decay	1e-5	Adam optimiser, L2 regularisation
Max epochs	50	Upper bound; early stopping usually ends training sooner
Early-stop patience	7 epochs	Epochs without a val PR-AUC gain before stopping

Hyperparameter	Value	Role
Early-stop min. delta	1e-4	Minimum val PR-AUC gain that resets patience
Focal loss gamma	2.0	Down-weights easy, well-classified examples
Focal loss alpha	0.25	Up-weights the minority fraud class
Train / val / test split	70% / 15% / 15%	Time-aware, stratified by TransactionDT

Appendix B: Additional Application Screenshots

Figure 4.2 already showed the live-assessment console. The two screenshots below show the application's other two pages: the Analysis and Findings page, which reproduces the architecture diagram, the four-model table, all three comparison figures, and the winning model's confusion matrix in one scrollable view; and the Reproducibility and Artifacts page, which exposes every trained checkpoint, the shared scaler, and the results table for direct download.

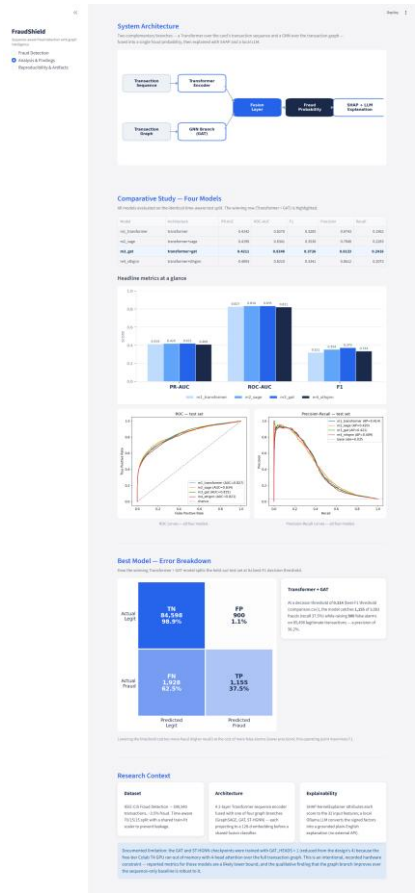

Figure A.1: The Analysis & Findings page — architecture diagram, the four-model comparison table and bar chart, ROC and precision-recall curves, the winning model's confusion matrix, and the honest GAT_HEADS limitation note, all in one place.

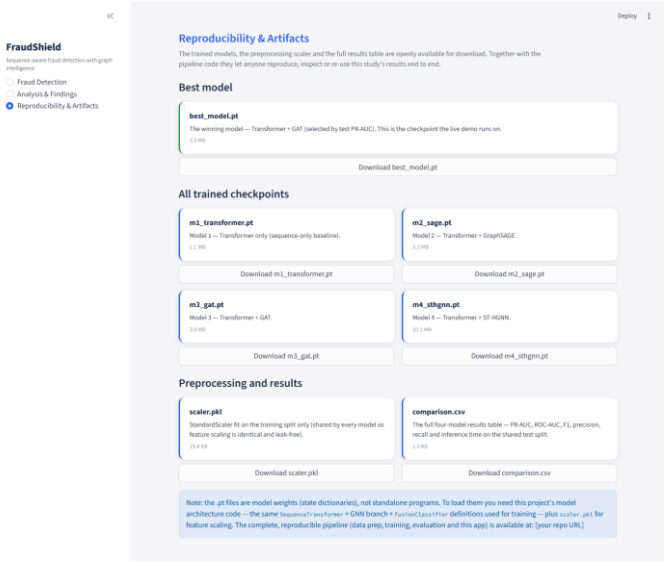

Figure A.2: The Reproducibility & Artifacts page, offering direct download of every trained checkpoint (m1-m4), the shared scaler, and the full results table.